Ejercicio 2.1. Simulación de la difusión del calor

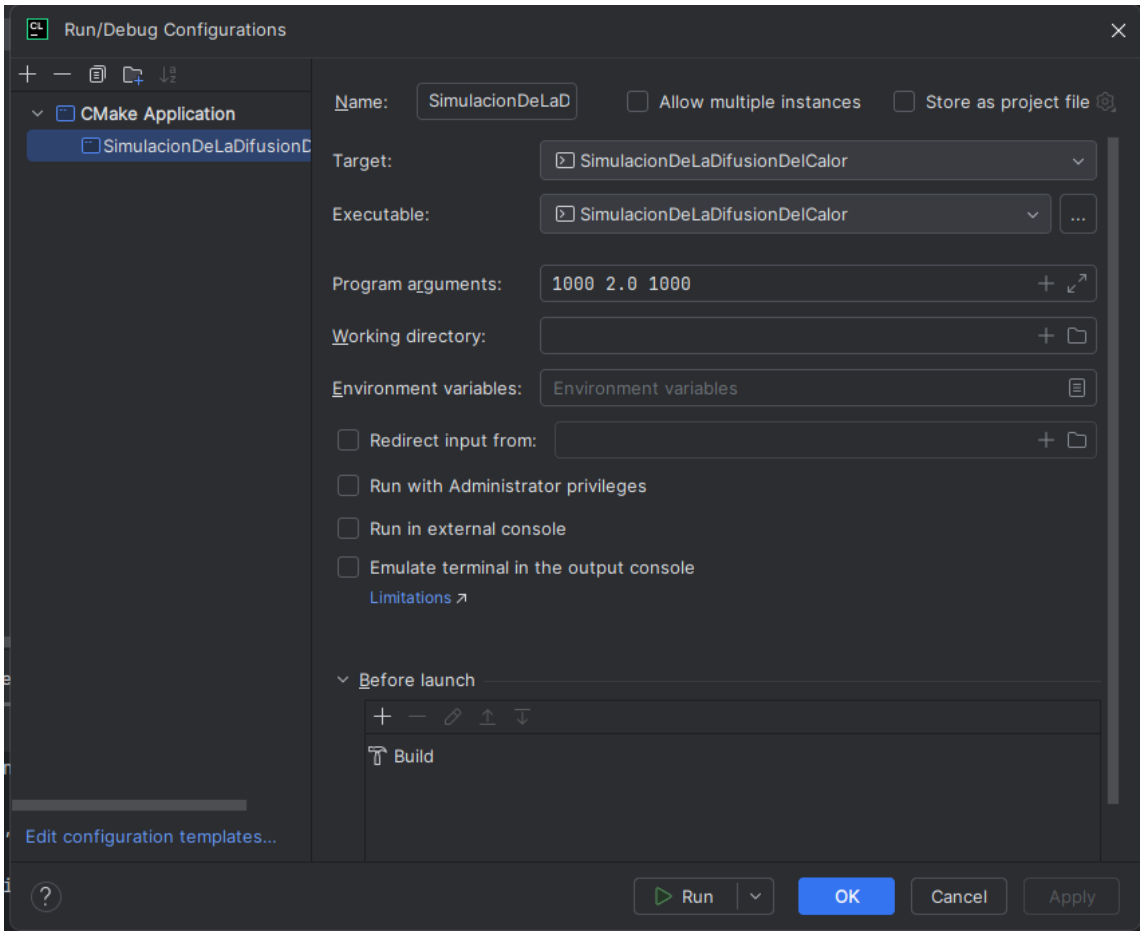
Alumnos: Rodrigo Yepes y Javier Gómez

1. Bloque de Capturas: Escalabilidad de N

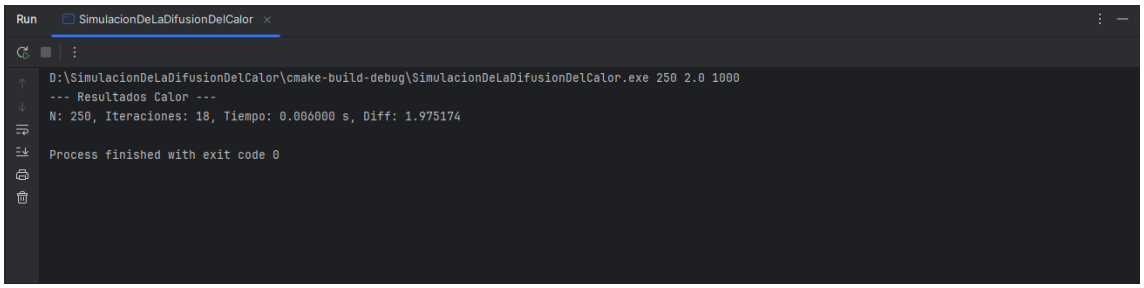
Se han realizado experimentos variando el valor de N y manteniendo fijos el umbral (2.0) y el número máximo de iteraciones (1000) :

N (Tamaño)	Tiempo Total (s)	Iteraciones
250	**0.006000 s**	18
500	**0.019000 s**	18
1000	**0.072000 s**	18

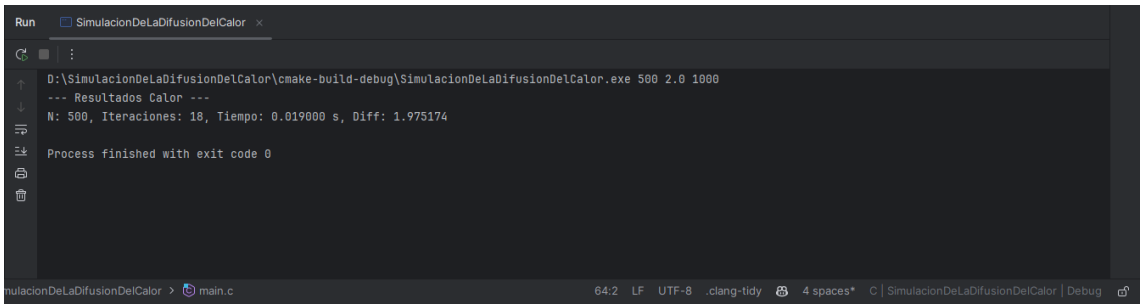
Justificación: Al aumentar N, el tiempo crece de forma cuadrática ($O(N^2)$), ya que el número total de elementos a procesar se multiplica por cuatro cada vez que doblamos N.



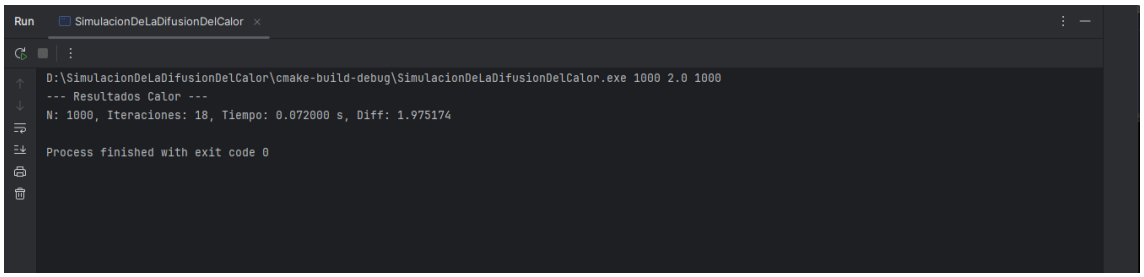
argumentos 250 2.0 1000:



argumentos 500 2.0 1000:



argumentos 1000 2.0 1000:



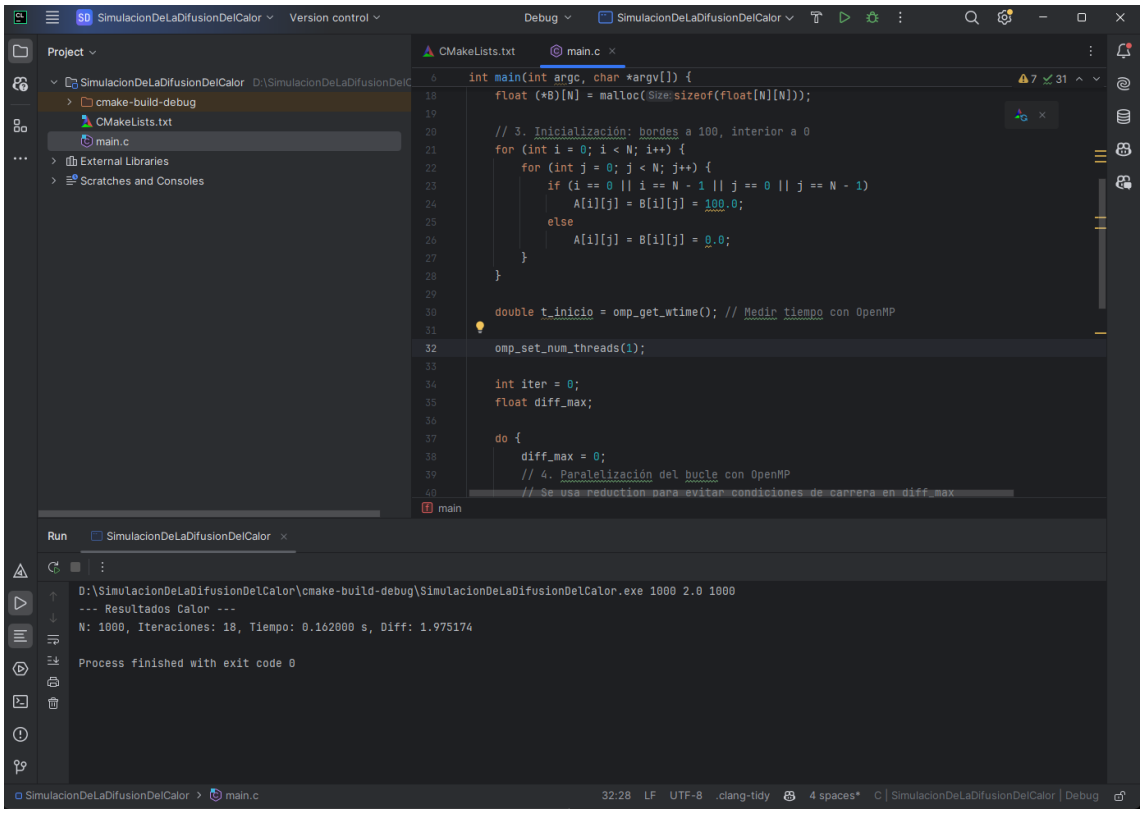
2. Bloque de Capturas: Escalabilidad de Hilos

Se ha variado el número de hilos mediante omp_set_num_threads() :

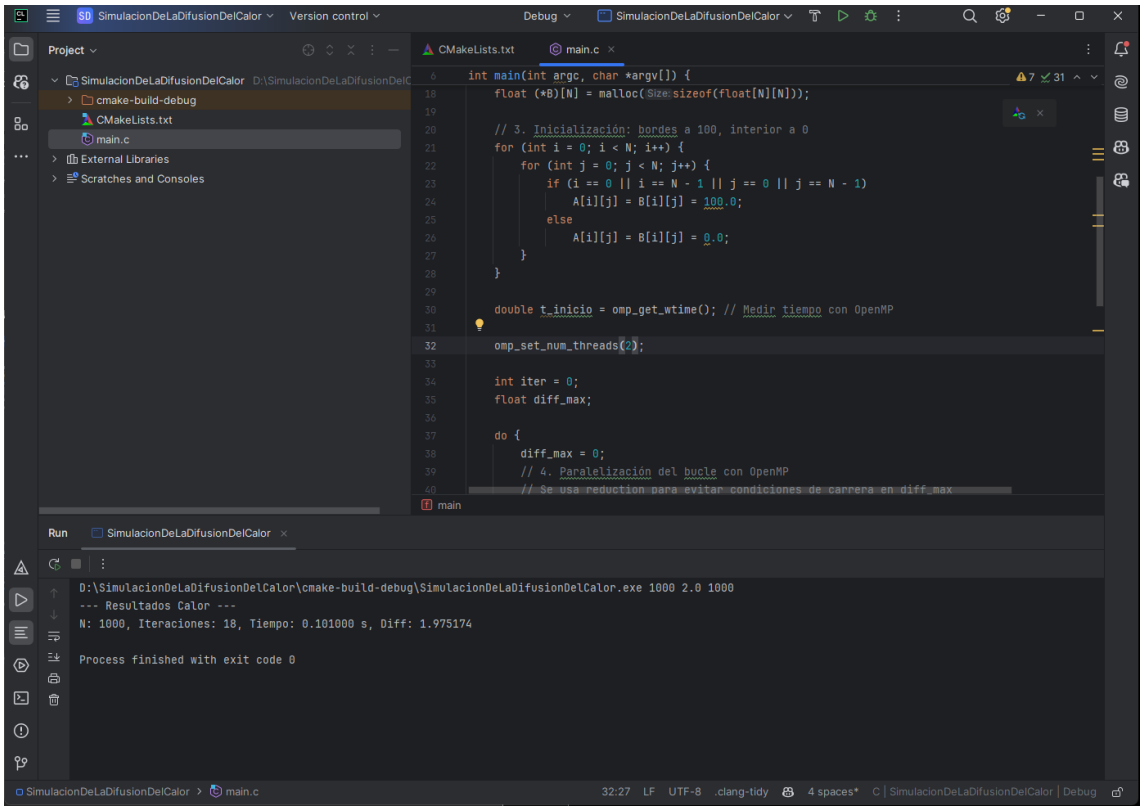
Nº Hilos	**Tiempo Total (s)**
1 hilo	**0.162000 s**
2 hilos	**0.080000 s**
8 hilos	**0.037000 s**

Justificación: El uso de paralelismo reduce significativamente el tiempo de ejecución. El Speedup es notable, aunque no es lineal debido al coste de gestión de hilos y la sincronización de la variable de reducción.

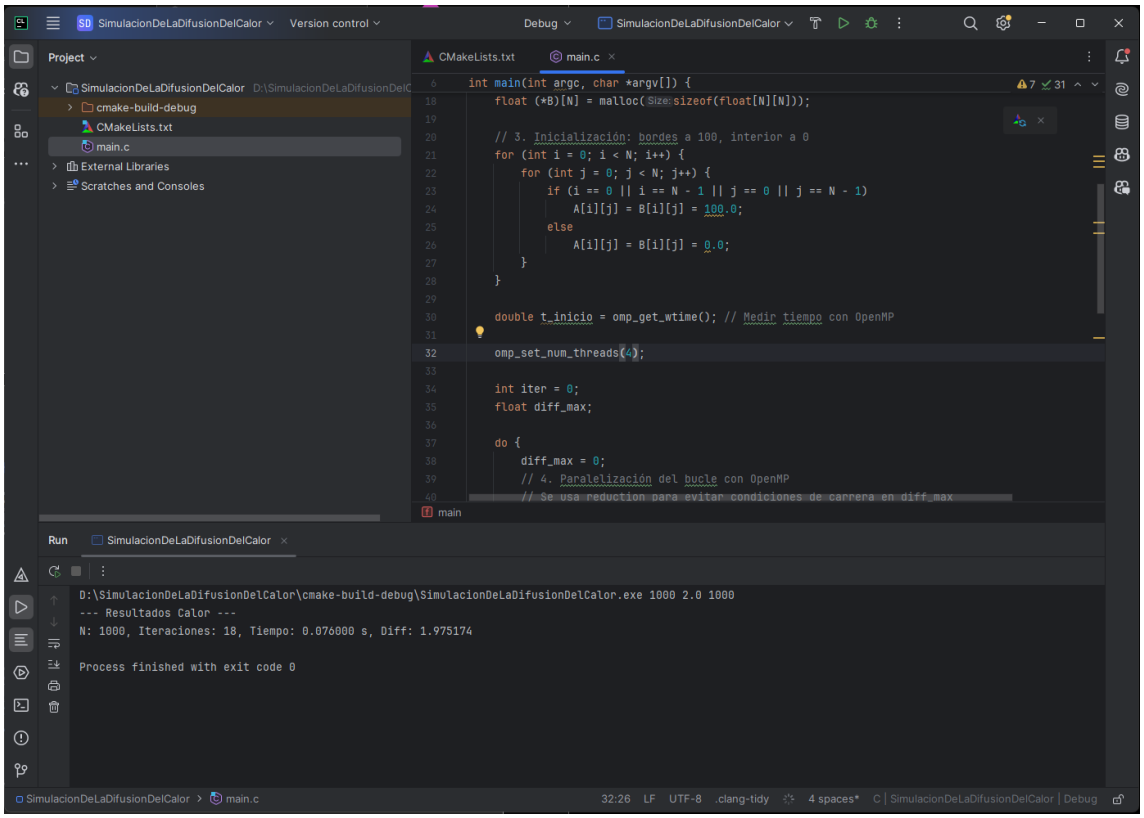
<u>1 hilo:</u>



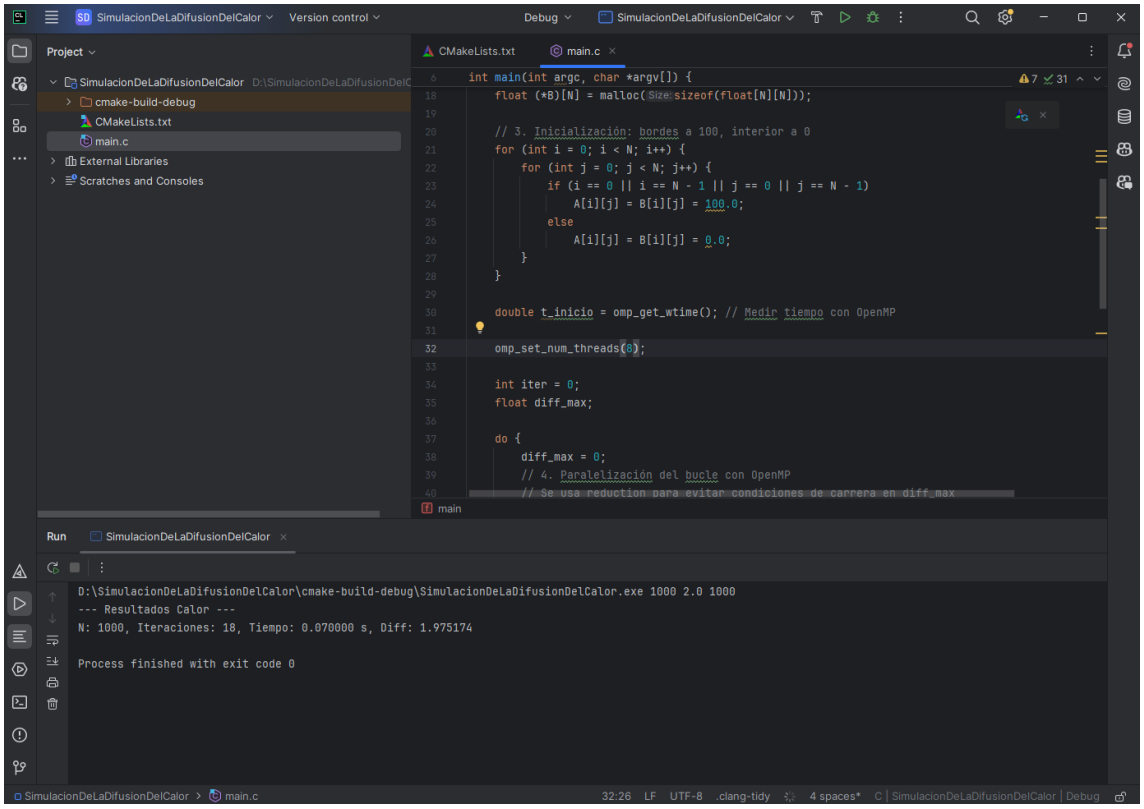
2 hilos:



4 hilos:



8 hilos:



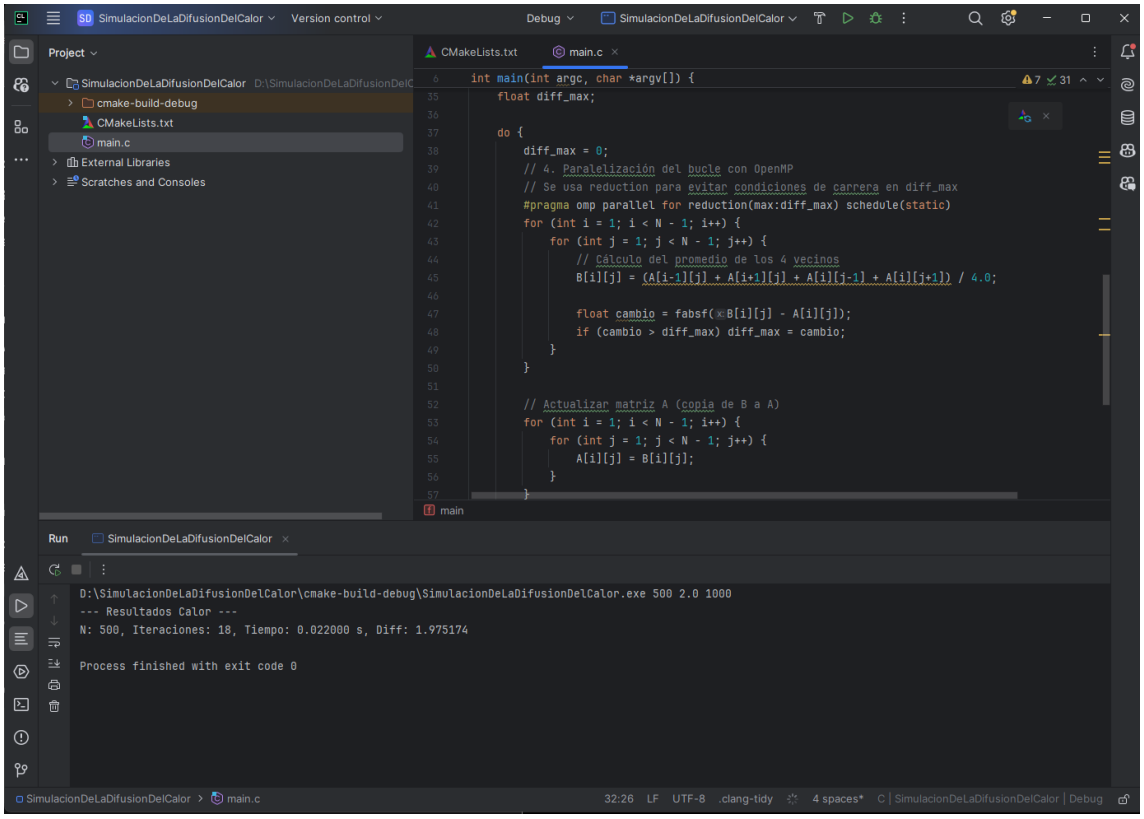
3. Bloque de Capturas: Sincronización (Reduction vs Critical)

Se han comparado los dos métodos para gestionar el cálculo del cambio máximo :

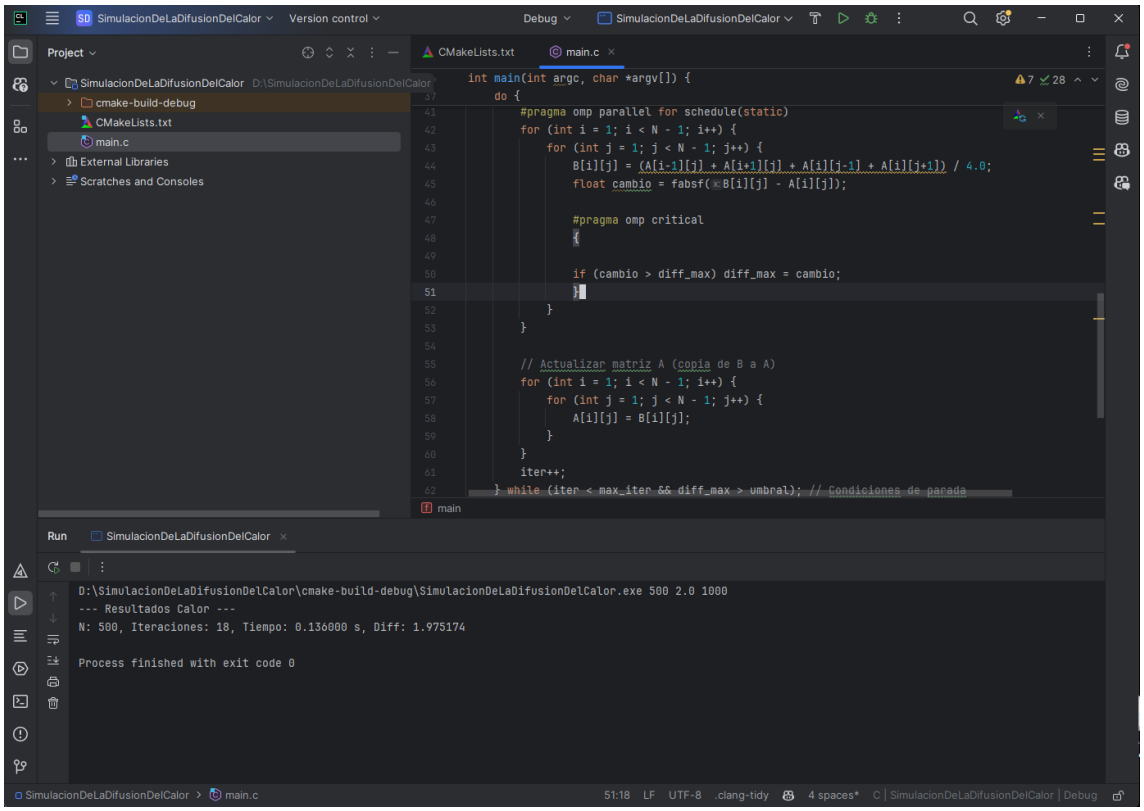
- **Reduction: 0.022000 s**
- **Critical: 0.136000 s**

Análisis: La directiva reduction permite que cada hilo mantenga su propia variable local, mientras que critical serializa el acceso, obligando a los hilos a esperar, lo que degrada drásticamente el rendimiento.

Paralelización con “reduction":



Paralelización con “critical"



4. Bloque de Capturas: Localidad de Memoria (Orden de bucles)

Comparación de Reparto (Schedule)

- **Static: 0.016000 s**
- **Dynamic: 0.130000 s Conclusión:** En este problema, la carga de trabajo es equilibrada en todas las filas. Por ello, el reparto estático es más eficiente al tener menos sobrecarga de gestión que el dinámico.

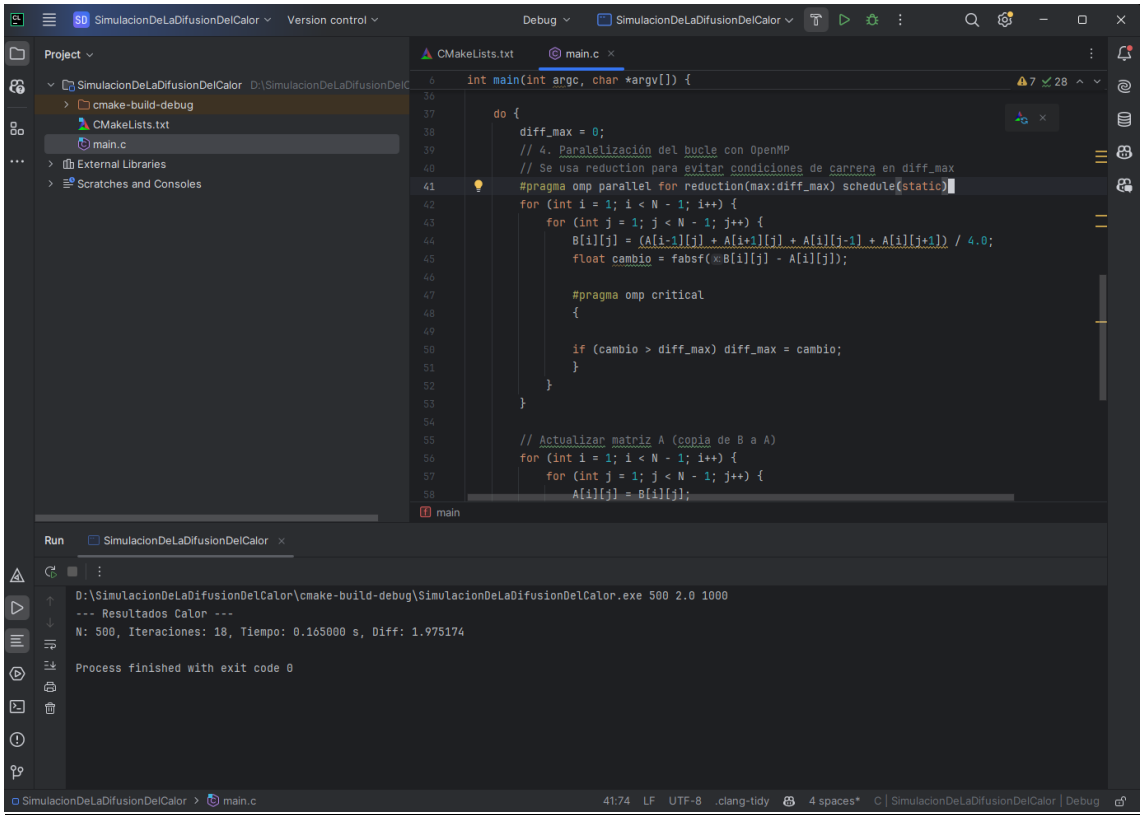
Inversión de Bucles (Localidad Espacial)

Al invertir el orden de los bucles (variando columnas en el exterior y filas en el interior), se observa una alteración en la eficiencia del acceso a memoria.

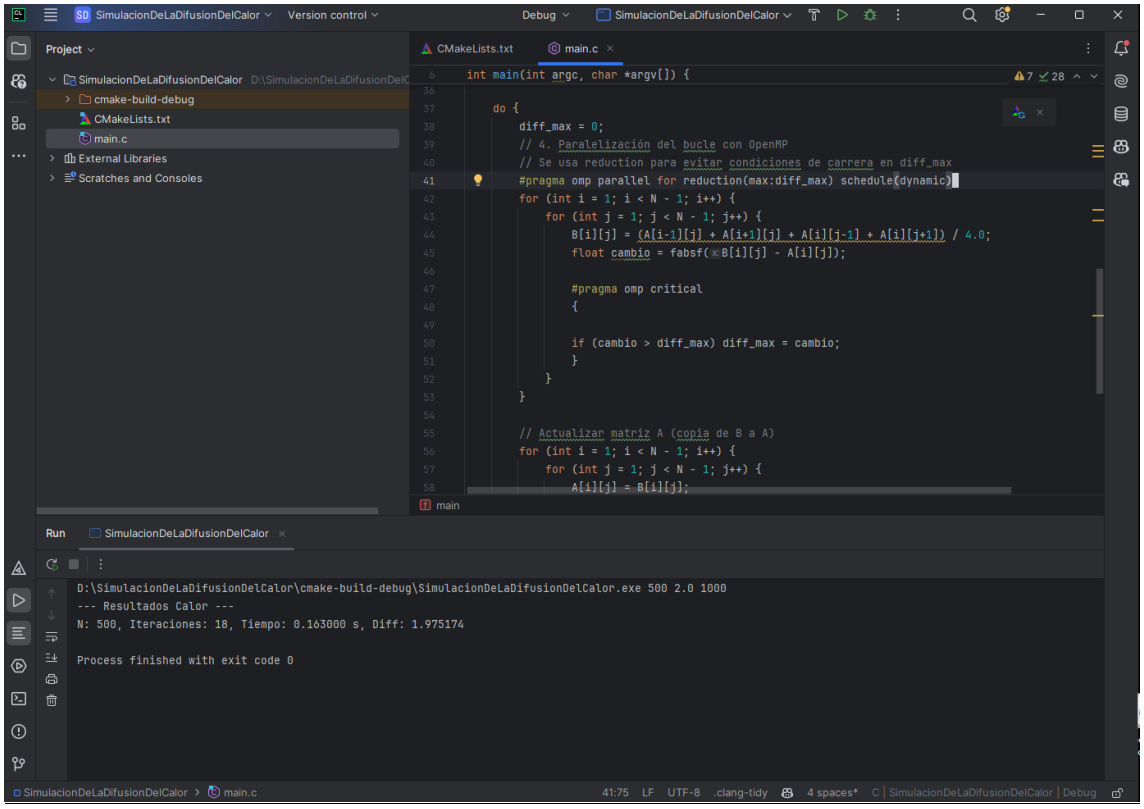
- **Orden Eficiente (Filas):** Accede a posiciones contiguas de memoria, aprovechando la **localidad espacial** y las líneas de la **caché L1**.
- **Orden Ineficiente (Columnas):** Provoca constantes **fallos de caché (cache misses)** al saltar entre filas, obligando a lecturas lentas desde la RAM.

Parte A: Comparación de Reparto (schedule)

version Static:

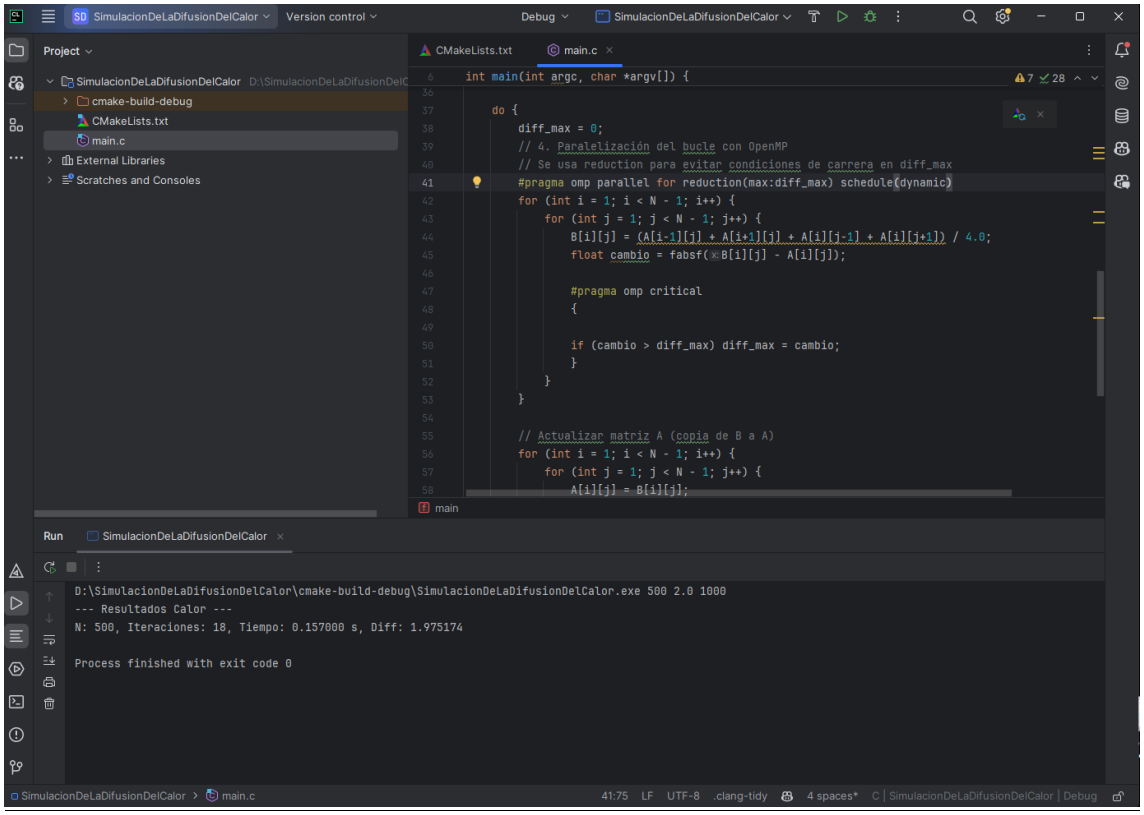


version Dynamic:

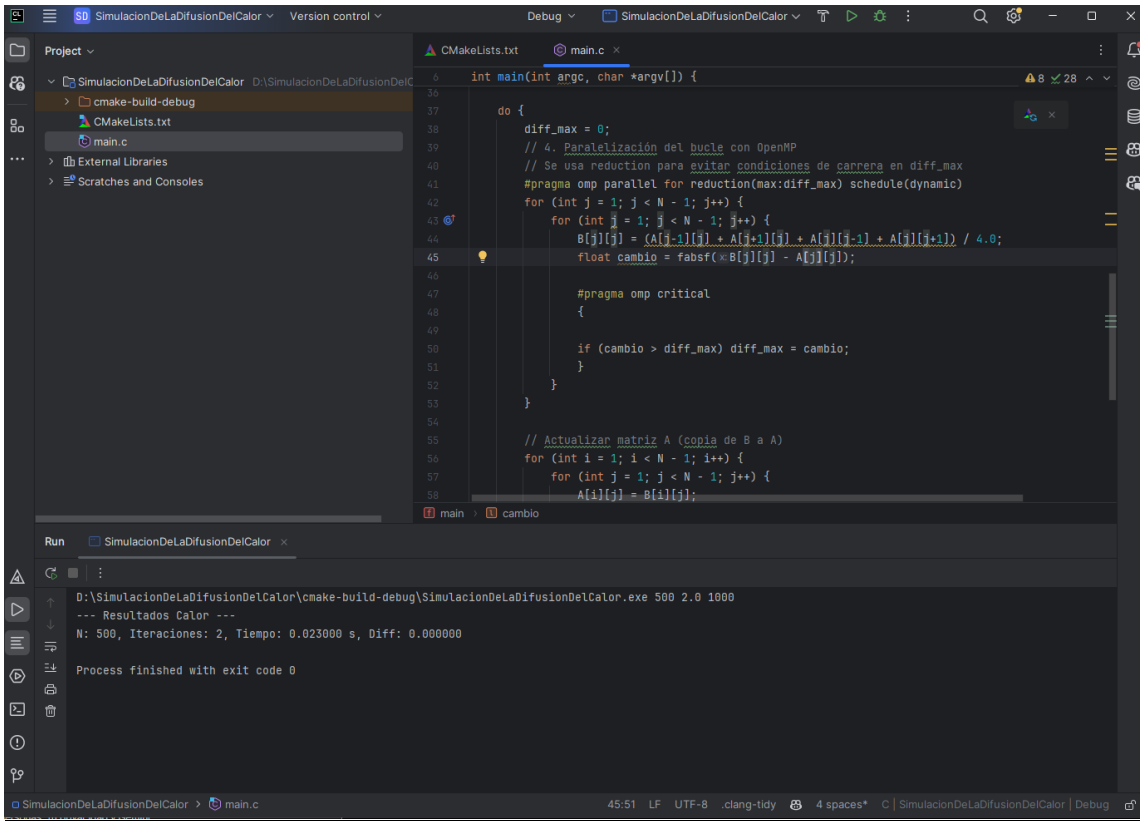


Parte B: Localidad de Memoria (Inversión de Bucles)

Código Normal (Eficiente): Primero i (filas), luego j (columnas).



Código Invertido (Ineficiente): Primero j (columnas), luego i (filas):



CÓDIGO USADO:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

int main(int argc, char *argv[]) {
  // 1. Lectura de argumentos: N, umbral e iteraciones
  if (argc < 4) {
    printf("Uso: %s <N> <umbral> <max_iter>\n", argv[0]);
    return 1;
  }
  int N = atoi(argv[1]);
  float umbral = atof(argv[2]);
  int max_iter = atoi(argv[3]);

  // 2. Alojamiento de memoria para matrices N x N
  float (*A)[N] = malloc(sizeof(float[N][N]));
  float (*B)[N] = malloc(sizeof(float[N][N]));

  // 3. Inicialización: bordes a 100, interior a 0
  for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
      if (i == 0 || i == N - 1 || j == 0 || j == N - 1)
        A[i][j] = B[i][j] = 100.0;
      else
        A[i][j] = B[i][j] = 0.0;
    }
  }

  double t_inicio = omp_get_wtime(); // Medir tiempo con OpenMP

  omp_set_num_threads(4);

  int iter = 0;
  float diff_max;

  do {
    diff_max = 0;
    // 4. Paralelización del bucle con OpenMP
```

```
    // Se usa reduction para evitar condiciones de carrera en diff_max
    #pragma omp parallel for schedule(static)
    for (int i = 1; i < N - 1; i++) {
      for (int j = 1; j < N - 1; j++) {
        B[i][j] = (A[i-1][j] + A[i+1][j] + A[i][j-1] + A[i][j+1]) / 4.0;

        float cambio = fabsf(B[i][j] - A[i][j]);
        if (cambio > diff_max) diff_max = cambio;
      }
    }

    // Actualizar matriz A (copia de B a A)
    for (int i = 1; i < N - 1; i++) {
      for (int j = 1; j < N - 1; j++) {
        A[i][j] = B[i][j];
      }
    }
    iter++;
  } while (iter < max_iter && diff_max > umbral); // Condiciones de parada

  double t_fin = omp_get_wtime();
  printf("--- Resultados Calor ---\n");
  printf("N: %d, Iteraciones: %d, Tiempo: %f s, Diff: %f\n", N, iter, t_fin - t_inicio,
diff_max);

  free(A); free(B);
  return 0;
}
```